MAKERERE UNIVERSITY | COLLEGE OF COMPUTING & INFORMATION SCIENCES

PLATFORMIO & VS CODE PROJECT STRUCTURE GUIDE

Multi-Target Directory Layout, Config Files & Git Workflow

Kayeera Nathan (24/U/25913/EVE) - System Architect (Group SE-24)

PLATFORMIO & VS CODE PROJECT STRUCTURE GUIDE

Project: Smart Control and Monitoring System for CoCIS (Makerere University)

Author: Kayeera Nathan (System Architect & Integrator, Group SE-24)

Framework: ESP-IDF (framework = espidf)

Target Platform: Espressif ESP32 (board = esp32dev)

Simulation: Wokwi VS Code Extension + Wokwi Web Simulator

Communication: Wireless ESP-NOW (2.4 GHz Peer-to-Peer)

1. Architectural Directory Layout (Monorepo with Multi-Environment)

This structure is designed specifically for PlatformIO in VS Code. It allows the entire 5-person team to collaborate seamlessly using Git without merge conflicts. The three firmware nodes are designated as ENTERANCE_MASTER, ROOM_NODE_1, and ROOM_NODE_2.

```
SE24_CoCIS_Smart_System/

    .vscode/                                # VS Code Workspace Configuration
        settings.json                       # Editor settings & PlatformIO path
        launch.json                         # Debugging configurations
        extensions.json                     # Recommended extensions (PlatformIO, Wokwi)

    .code-workspace                         # Multi-root VS Code launcher file
    .gitignore                              # Exclude .pio, build artifacts, temp files
    platformio.ini                          # Central multi-environment PlatformIO config
    wokwi.toml                              # Wokwi VS Code extension configuration
    diagram.json                            # 3-ESP32 Wokwi circuit simulation schematic
    CMakeLists.txt                          # Root ESP-IDF CMakeLists

    components/                             # Shared ESP-IDF Components & Libraries
        common_protocol/                   # Shared Packet Definitions & Concurrency Headers
            CMakeLists.txt                  # ESP-IDF component registration
            include/
                protocol.h                  # ESP-NOW packet frames & message types
                sync_objects.h              # FreeRTOS mutexes & queue handles
                config_defaults.h          # Default timings, pin maps & thresholds
                mac_address_map.h           # Virtual MAC addresses of all 3 nodes
            src/
                protocol.c                  # Checksum calculation & packet serialization
                sync_objects.c              # Mutex & queue initializer helpers

    src/                                    # Firmware Source Code (Split by Node)

        ENTERANCE_MASTER/                   # ENTERANCE_MASTER: Entrance Terminal & Gateway (E
                                            # Owners: Nathan (Lead), David (Entrance), Jovan (
            main.c                          # Master app_main(), task spawning, system state
            rfid_manager.c                  # SPI RC522 reader handling (Entry & Exit)
```

```
        rfid_manager.h
        door_actuator.c                 # Servo PWM control & 5s timed relock
        door_actuator.h
        schedule_enforcer.c             # 07:00-20:00 schedule check & curfew lockdown
        schedule_enforcer.h
        anti_passback.c                 # State-based anti-passback validation
        anti_passback.h
        sd_storage.c                    # FATFS file operations (users, logs, config)
        sd_storage.h
        serial_cli.c                    # Administrative terminal parser & menus
        serial_cli.h
        auth_security.c                 # Password hashing & session timeout
        auth_security.h
        esp_now_gateway.c               # ESP-NOW receiver callback & heartbeat monitor
        esp_now_gateway.h

    ROOM_NODE_1/                        # ROOM_NODE_1: Lecture Room 1 Terminal (ESP32 #2)
                                        # Owner: Zziwa Mark Alex
        main.c                          # Room 1 app_main() & event loop
        motion_sensor.c                 # PIR interrupt, debounce & 30s countdown
        motion_sensor.h
        light_sensor.c                  # ADC LDR reading & daylight-aware thresholding
        light_sensor.h
        light_relay.c                   # GPIO relay control & manual override
        light_relay.h
        esp_now_sender.c                # 2-second telemetry telemetry dispatch
        esp_now_sender.h

    ROOM_NODE_2/                        # ROOM_NODE_2: Lecture Room 2 Terminal (ESP32 #3)
                                        # Owner: Zziwa Mark Alex
        main.c                          # Room 2 app_main() & event loop
        motion_sensor.c                 # Modular PIR driver
        motion_sensor.h
        light_sensor.c                  # Modular LDR driver
        light_sensor.h
        light_relay.c                   # Modular Relay driver
        light_relay.h
        esp_now_sender.c                # Telemetry sender with Room 2 MAC/ID
        esp_now_sender.h

simulation/                             # Simulation Assets & Virtual Storage
    sdcard/                             # Pre-populated virtual MicroSD files
        users.csv                       # Registered tags (Nathan, David, Mark, Jovan, Lau
        access_log.csv                  # Historical audit log sample
        occupancy.csv                   # Crash recovery state file
        config.txt                      # Initial system configuration

docs/                                   # Documentation, Schematics & Report Drafts
    architecture/                       # SYSTEM_ARCHITECTURE_DIAGRAM.svg & .png
    interface_contract/                 # INTERFACE_CONTRACT_AND_DESIGN_WIRELESS.pdf
    test_matrix/                        # Test cases & bug tracking spreadsheet
    report/                             # LaTeX / Word report source & member contribution
```

2. Core Configuration Files (Copy-Paste Ready)

2.1 ‘platformio.ini‘ (Central Multi-Target Configuration)

Place this file in the root directory. It configures PlatformIO to build three distinct firmware binaries: ENTERANCE_MASTER, ROOM_NODE_1, and ROOM_NODE_2:

```
; ========================================================================
; PlatformIO Project Configuration for SE-24 CoCIS Smart System
; Multi-Target ESP-IDF Setup: ENTERANCE_MASTER, ROOM_NODE_1, ROOM_NODE_2
; ========================================================================

[platformio]
default_envs = ENTERANCE_MASTER

[env]
platform = espressif32 @ 6.5.0
framework = espidf
board = esp32dev
monitor_speed = 115200
monitor_filters = direct, esp32_exception_decoder

; ------------------------------------------------------------------------
; Target 1: ENTERANCE_MASTER - Entrance Master & Gateway (ESP32 #1)
; ------------------------------------------------------------------------
[env:ENTERANCE_MASTER]
build_src_filter = +<ENTERANCE_MASTER/*>
build_flags =
    -D NODE_ENTERANCE_MASTER=1
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include

; ------------------------------------------------------------------------
; Target 2: ROOM_NODE_1 - Lecture Room 1 Terminal (ESP32 #2)
; ------------------------------------------------------------------------
[env:ROOM_NODE_1]
build_src_filter = +<ROOM_NODE_1/*>
build_flags =
    -D NODE_ROOM_1=1
    -D ROOM_ID=1
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include

; ------------------------------------------------------------------------
; Target 3: ROOM_NODE_2 - Lecture Room 2 Terminal (ESP32 #3)
; ------------------------------------------------------------------------
[env:ROOM_NODE_2]
build_src_filter = +<ROOM_NODE_2/*>
build_flags =
    -D NODE_ROOM_2=1
    -D ROOM_ID=2
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include
```

2.2 ‘wokwi.toml‘ (Multi-Node Wokwi Simulation Config)

Place this in the root directory. It connects the VS Code Wokwi extension to your build outputs:

```
[wokwi]
version = 1
diagram = "diagram.json"
firmware = ".pio/build/ENTERANCE_MASTER/firmware.bin"
elf = ".pio/build/ENTERANCE_MASTER/firmware.elf"

# Multi-node simulation chips:
[[wokwi.chips]]
id = "enterance_master_esp"
firmware = ".pio/build/ENTERANCE_MASTER/firmware.bin"
elf = ".pio/build/ENTERANCE_MASTER/firmware.elf"
```

```
[[wokwi.chips]]
id = "room_node_1_esp"
firmware = ".pio/build/ROOM_NODE_1/firmware.bin"
elf = ".pio/build/ROOM_NODE_1/firmware.elf"

[[wokwi.chips]]
id = "room_node_2_esp"
firmware = ".pio/build/ROOM_NODE_2/firmware.bin"
elf = ".pio/build/ROOM_NODE_2/firmware.elf"
```

2.3 ‘diagram.json‘ (3-Board Cable-Free Wokwi Circuit)

Because we use ESP-NOW, there are no physical serial wires between the boards:

```
{
  "version": 1,
  "author": "Kayeera Nathan (Group SE-24)",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "enterance_master_esp", "top": 0, "left": 0, "at
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_1_esp",      "top": 0, "left": 420, "
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_2_esp",      "top": 0, "left": 840, "

    { "type": "wokwi-servo", "id": "servo_entry", "top": 180, "left": -120, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo_exit",  "top": 260, "left": -120, "attrs": {} },
    { "type": "wokwi-led",   "id": "led_green",   "top": 350, "left": -80,  "attrs": { "color":
    { "type": "wokwi-led",   "id": "led_red",     "top": 350, "left": -40,  "attrs": { "color":
    { "type": "wokwi-buzzer","id": "buzzer1",     "top": 400, "left": -80,  "attrs": {} },

    { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": 180, "left": 400, "attrs": {} },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr1", "top": 260, "left": 400, "attrs": {}
    { "type": "wokwi-relay-module", "id": "relay1", "top": 340, "left": 400, "attrs": {} },

    { "type": "wokwi-pir-motion-sensor", "id": "pir2", "top": 180, "left": 820, "attrs": {} },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr2", "top": 260, "left": 820, "attrs": {}
    { "type": "wokwi-relay-module", "id": "relay2", "top": 340, "left": 820, "attrs": {} }
  ],
  "connections": [
    [ "enterance_master_esp:13", "servo_entry:PWM", "orange", [ "v0" ] ],
    [ "enterance_master_esp:15", "servo_exit:PWM",  "orange", [ "v0" ] ],
    [ "enterance_master_esp:2",  "led_green:A",     "green",  [ "v0" ] ],
    [ "enterance_master_esp:4",  "led_red:A",       "red",    [ "v0" ] ],
    [ "enterance_master_esp:12", "buzzer1:1",       "purple", [ "v0" ] ],

    [ "room_node_1_esp:14",      "pir1:OUT",        "blue",   [ "v0" ] ],
    [ "room_node_1_esp:34",      "ldr1:AO",         "gold",   [ "v0" ] ],
    [ "room_node_1_esp:12",      "relay1:IN",       "green",  [ "v0" ] ],

    [ "room_node_2_esp:14",      "pir2:OUT",        "blue",   [ "v0" ] ],
    [ "room_node_2_esp:34",      "ldr2:AO",         "gold",   [ "v0" ] ],
    [ "room_node_2_esp:12",      "relay2:IN",       "green",  [ "v0" ] ]
  ]
}
```

2.4 ‘CMakeLists.txt‘ for ESP-IDF in PlatformIO

Root ‘CMakeLists.txt‘:

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(cocis_smart_system)
```

‘components/common_protocol/CMakeLists.txt‘:

```
idf_component_register(
    SRCS "src/protocol.c" "src/sync_objects.c"
    INCLUDE_DIRS "include"
    REQUIRES nvs_flash esp_wifi esp_netif esp_now freertos
)
```

2.5 ‘.gitignore‘ (Keep the Repo Clean)

```
.pio/
.vscode/.browse.c_cpp.db*
.vscode/c_cpp_properties.json
build/
*.bin
*.elf
*.map
__pycache__/
*.pyc
.DS_Store
Thumbs.db
```

3. Team Member Directory & Git Ownership Matrix

Member	Assigned Directory / Files	Git Branch	Core Responsibilities
Kayeera Nathan	components/common_protocol/, src/ENTERANCE_MASTER/main.c, esp_now_gateway.*, platformio.ini, wokwi.toml	main, feat/arch-nathan	System architecture, FreeRTOS queues/mutexes, ESP-NOW protocol, emergency mode, integrator.
Kasozi David Paul	src/ENTERANCE_MASTER/rfid_manager.*, door_actuator.*, anti_passback.*	feat/entrance-david	RFID read drivers, servo door opening, anti-passback rule, intrusion lockout.
Zziwa Mark Alex	src/ROOM_NODE_1/, src/ROOM_NODE_2/	feat/rooms-mark	PIR motion sensing, 30s timer, LDR daylight-aware lighting, relay actuators, telemetry sender.
Sseku Jovan	src/ENTERANCE_MASTER/sd_storage.*, simulation/sdcard/	feat/storage-jovan	SPI MicroSD driver, FATFS mounting, CSV logs, crash recovery from reboot.
Namuzibwa Laurinda	src/ENTERANCE_MASTER/serial_cli.*, auth_security.*, docs/test_matrix/	feat/cli-laurinda	Command parser loop, SHA-256 login, schedule set command, test plan & QA.

4. How to Open & Build in VS Code (Step-by-Step)

- 1. Install Prerequisites in VS Code:
- Install PlatformIO IDE extension (platformio.platformio-ide).
- Install Wokwi Simulator extension (wokwi.wokwi-vscode).
- 2. Open Project Folder:
- Launch VS Code $\rightarrow$ File $\rightarrow$ Open Folder... $\rightarrow$ Select SE24_CoCIS_Smart_System.

- 3. Compile a Specific Target:
- Click the PlatformIO Alien icon on the left sidebar.
- Under Project Tasks, select:
- ENTERANCE_MASTER $\rightarrow$ Click Build
- ROOM_NODE_1 $\rightarrow$ Click Build
- ROOM_NODE_2 $\rightarrow$ Click Build
- Or run in terminal: pio run -e ENTERANCE_MASTER
- 4. Run Live Wokwi Simulation:
- Press F1 (or Ctrl+Shift+P) in VS Code.
- Type Wokwi: Start Simulator and press Enter.
- Wokwi will load diagram.json and start all three ESP32 microcontrollers in browser simulation with live wireless communication!